

POSIX Threads

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

Departamento de Engenharia Informática

POSIX Threads

Threads: Lightweight Execution

- A thread is an independent flow of execution inside a process.
- While processes provide the container and isolation, threads represent the active paths of logic being executed.
 - ▶ **Resource Sharing:** Threads within the same process share the process address space (code, data, and files).
 - ▶ **Independent Scheduling:** Each thread is scheduled independently by the OS kernel.
 - ▶ **Concurrency:** Threads enable true concurrency in multi-core systems, allowing multiple tasks to execute in parallel.

Advantages of Multithreading

- **Structured execution:** Facilitates structuring programs into multiple independent units of execution.
- **Memory efficiency:** Eliminates the need for multiple address spaces, reducing memory overhead.
- **Synchronization:** Allows for more efficient and lightweight synchronization mechanisms compared to inter-process communication.
- **Context switching:** Changing context between threads is significantly faster than between processes, as the address space remains the same.

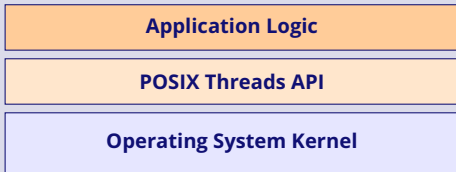
Disadvantages and Challenges

Multithreading introduces increased complexity in shared resource management.

- **Data Consistency:** Shared resources require careful management, as changes made by one thread are immediately visible to all others.
- **Programmer Responsibility:** Synchronization of shared data (e.g., variables in the heap) rests entirely on the programmer.
- **Debugging Difficulty:** The lack of memory isolation can lead to non-deterministic bugs, such as race conditions and deadlocks, which are notoriously difficult to reproduce and fix.

POSIX Threads (*aka* Pthreads)

- Standardised API, defined by the IEEE POSIX 1003.1c standard.
- Provides a unified, portable interface for thread management across different operating systems.
- Decouples application logic from the underlying kernel thread implementation, ensuring code portability.



Building Pthread Applications

- The Pthreads library is not part of the standard C library (libc) on most Linux distributions.
- **GCC (GNU Compiler Collection):**
 - ▶ Must explicitly link the library using the `-lpthread` flag:
 - ▶ `gcc -o my_app main.c -lpthread`
- **Clang/LLVM:**
 - ▶ On many modern systems (e.g., macOS), the Pthreads library is integrated into the default system library, so explicit linking is often unnecessary.
 - ▶ `clang -o my_app main.c`
- **Best Practice:** Always verify linking requirements for your specific target environment.

Thread Creation: pthread_create()

pthread_create() Signature

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine)(void *), void *arg);
```

- **Parameters:**

- ▶ thread: Pointer to the thread identifier (handle).
- ▶ attr: Thread attributes (use NULL for default).
- ▶ start_routine: Pointer to the function to be executed.
- ▶ arg: Single argument passed to the start function.

- **Return Value:** Returns 0 on success; an error number otherwise.

Understanding pthread_create() Arguments

- **Thread Identifier (pthread_t)**

- ▶ Each thread is assigned a unique identifier of type pthread_t.
- ▶ This handle is used to reference the thread in subsequent operations (e.g., joining or cancelling).

- **Thread Attributes (pthread_attr_t)**

- ▶ A structure used to configure thread properties, such as stack size, scheduling policy, and detachment state.
- ▶ Allows developers to influence how the system manages the thread before it is created.

Understanding pthread_create() Arguments

- **Start Routine Signature**

- ▶ The function executed by the thread must strictly adhere to the following signature:

```
void* routine_name(void* arg);
```

- ▶ This generic pointer approach allows for flexible data passing between the main process and the thread.

- **Thread Argument (void* arg)**

- ▶ A generic pointer used to pass a single argument to the thread function.
- ▶ Since it is a void *, you can pass the address of any data structure (e.g., structs, arrays, or scalars), allowing complex data to be shared with the new thread.

Practical Example: Creating and Joining a Thread

```
#include <stdio.h>
#include <pthread.h>

void *my_task(void *arg) {
    printf("[NEW THREAD] I am the new thread!\n");
    return NULL;
}

int main() {
    pthread_t thread_id;

    pthread_create(&thread_id, NULL, my_task, NULL);

    printf("[MAIN THREAD] Created new thread %lu.\n", (unsigned long)thread_id);
    pthread_join(thread_id, NULL);
    return 0;
}
```

Execution Output and Verification

```
user@machine PRASIE % clang T2-01.c -o T2-01
user@machine PRASIE % ./T2-01
[MAIN THREAD] Created new thread 123145386291200.
[NEW THREAD] I am the new thread!
user@machine PRASIE %
```

Important Note

The interleaving of output lines is non-deterministic. The order of execution depends on the OS thread scheduler and may vary between runs.

Wait for Thread Termination: pthread_join()

pthread_join() Signature

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **retval);
```

- **Parameters:**

- ▶ thread: The unique identifier of the target thread to wait for.
- ▶ retval: A pointer to a location where the target thread's exit value is stored. Pass NULL if the return value is not needed.

- **Return Value:**

- ▶ Returns 0 on success; an error number otherwise.

Behaviour of pthread_join()

- **Suspension:** The calling thread is suspended until the target thread terminates. If the target thread has already finished, the function returns immediately.
- **Data Retrieval:** If `retval` is not NULL, the exit status returned by the thread is stored at the memory address referenced by `retval`.
- **Completion:** Upon a successful return, the target thread is guaranteed to have been terminated and its resources reclaimed.
- **Constraints and Undefined Behaviour:**
 - ▶ **Multiple Joins:** Calling `pthread_join()` multiple times simultaneously for the same target thread results in *undefined behaviour*.
 - ▶ **Cancellation:** If the calling thread is cancelled, the target thread remains attached (not detached).

Example: Returning and Retrieving Values (1/2)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *my_task(void *arg) {
    int *result = malloc(sizeof(int)); // Heap allocation (Note: avoid in critical/embedded systems)
    *result = 42;
    return (void *)result; // Return pointer to memory
}

// continued on next slide...
```

Example: Returning and Retrieving Values (2/2)

```
// ... continued from previous slide

int main() {
    pthread_t tid;
    int *res;
    pthread_create(&tid, NULL, my_task, NULL);

    // Retrieve the pointer address returned by the thread
    pthread_join(tid, (void **)&res);

    printf("Thread %lu returned: %d\n", (unsigned long)tid, *res);
    free(res); // Clean up allocated memory
    return 0;
}
```

Execution Output and Verification

```
user@machine PRASIE % ./T2-02  
Thread 123145530077184 returned: 42  
user@machine PRASIE %
```

Memory Management in Embedded Systems

Warning: Dynamic Memory in Embedded Systems

- **Usage in Embedded Systems:** The use of `malloc()` is generally discouraged due to fragmentation risks and non-deterministic execution times.
- **Safety-Critical Systems:** In many safety-critical applications (e.g., automotive or aerospace standards like MISRA C), dynamic memory allocation is strictly prohibited to ensure predictable behavior and prevent memory exhaustion.
- **Alternative:** Use static allocation (global/static variables) or pre-allocated memory pools whenever possible.

Thread Termination: pthread_exit()

pthread_exit() Signature

```
#include <pthread.h>
```

```
void pthread_exit(void *retval);
```

- **Parameter:**

- ▶ **retval:** A pointer to an exit status value. This value is made available to any thread that performs a pthread_join() on this thread.

- **Purpose:** Terminates the calling thread immediately.

Using `pthread_exit()`

- **Behavior:**

- ▶ Unlike `return`, calling `pthread_exit()` ends the thread regardless of whether you are in the start routine or a function called by it.
- ▶ Does not terminate the entire process (unless it is the last running thread).

- **Why use it over `return`?**

- ▶ `return` only exits the current function. If your thread calls several nested functions, `pthread_exit()` provides a clean way to terminate from anywhere in the call stack.

- **Implicit Termination:**

- ▶ If the main thread calls `pthread_exit()` instead of `return 0` in `main()`, the process will continue to run as long as other threads are still active.

- **Resource Safety:**

- ▶ Ensures that thread-specific cleanup handlers are executed before termination.

Thread Identification: pthread_self()

pthread_self() Signature

```
#include <pthread.h>
```

```
pthread_t pthread_self(void);
```

- **Purpose:** Returns the unique identifier (pthread_t) of the calling thread.
- **Usage:**
 - ▶ Essential for threads to perform operations on themselves (e.g., self-detachment).
 - ▶ Often used for logging, debugging, or comparing thread handles to ensure that a specific logic is being executed by the correct thread.
- **Return Value:** Always succeeds; returns the ID of the thread that called it.

When to use pthread_self()

- **Logging and Tracing:**

- ▶ Helps differentiate logs from multiple threads executing the same function:

```
printf("Thread %lu is processing data\n", (unsigned long)pthread_self());
```

- **Self-Identification:**

- ▶ Useful when a thread needs to verify its own identity within a complex structure (e.g., in a thread pool where the thread must identify its index in an array).

- **Comparison:**

- ▶ Used with pthread_equal(t1, t2) to check if two thread identifiers refer to the same thread.

Thread Comparison: pthread_equal()

pthread_equal() Signature

```
#include <pthread.h>
```

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

- **Purpose:** Compares two thread identifiers to determine if they refer to the same thread.
- **Return Value:**
 - ▶ Returns a non-zero value (true) if the threads are identical.
 - ▶ Returns zero (false) if they are different.
- **Important:** The behavior is undefined if either thread ID is invalid.

Why pthread_equal() is Necessary

- **Opaque Type:** The pthread_t type is defined as an *opaque object*. Its internal representation (e.g., struct, pointer, or integer) is implementation-dependent.
- **Non-Portable Comparisons:**
 - ▶ Using the equality operator (==) directly is non-portable.
 - ▶ On some systems, == might work if the ID is an integer, but on others (where the ID might be a complex structure), it will fail or cause compilation errors.
- **Best Practice:** Always use pthread_equal() to ensure your code remains portable across different POSIX-compliant operating systems.

Common Pitfalls and Best Practices

The `main()` Thread and Process Termination

- **Implicit Termination:**

- ▶ If the `main()` function returns (or calls `exit()`), the entire process terminates.
- ▶ This occurs regardless of whether other threads are still running.

- **The Consequence:**

- ▶ All active threads are abruptly killed by the kernel when the process terminates.
- ▶ Worker threads will not have the chance to finish their tasks, perform cleanup, or store results.

Example of Implicit Termination (1/2)

```
#include <stdio.h>
#include <pthread.h>

#define NUM_TASKS 5

void *my_task(void *arg) {
    printf("[THREAD %lu] I'm alive!\n", (unsigned long)pthread_self());
    pthread_exit(NULL);
}

// continued on next slide...
```

Example of Implicit Termination (2/2)

```
// ... continued from previous slide

int main() {
    pthread_t thread_id[NUM_TASKS];

    for(int i = 0; i < NUM_TASKS; i++) {
        pthread_create(&thread_id[i], NULL, my_task, NULL);
    }

    printf("[MAIN THREAD %lu] Main task completed. Exiting.\n", (unsigned long)pthread_self());
    return 0;
}
```

Execution Output and Verification

```
user@machine PRASIE % ./T2-03  
[THREAD 123145327783936] I'm alive!  
[MAIN THREAD 140704388616576] Main task completed. Exiting.  
[THREAD 123145328857088] I'm alive!  
user@machine PRASIE %
```

Best Practices for Process Termination

- **Solution 1: Wait for Threads**

- ▶ Before the `main()` function reaches its end, use `pthread_join()` for every joinable thread you have created.
- ▶ This ensures the `main()` thread blocks until worker threads finish naturally.

- **Solution 2: Exit main thread before returning**

- ▶ If you need the `main()` function to exit without killing other threads, use `pthread_exit()` at the end of `main()`.
- ▶ This allows the process to stay alive as long as at least one thread remains active.

Implementing Solution 1: Thread Joining

```
int main() {
    pthread_t thread_id[NUM_TASKS];

    for(int i = 0; i < NUM_TASKS; i++) {
        pthread_create(&thread_id[i], NULL, my_task, NULL);
    }

    printf("[MAIN THREAD %lu] Main task completed. Exiting.\n", (unsigned long)pthread_self());

    // Thread joining.
    for(int i = 0; i < NUM_TASKS; i++) {
        pthread_join(thread_id[i], NULL);
    }

    return 0;
}
```

Implementing Solution 2: Main Thread Termination

```
int main() {
    pthread_t thread_id[NUM_TASKS];

    for(int i = 0; i < NUM_TASKS; i++) {
        pthread_create(&thread_id[i], NULL, my_task, NULL);
    }

    printf("[MAIN THREAD %lu] Main task completed. Exiting.\n", (unsigned long)pthread_self());

    // Main thread termination.
    pthread_exit(NULL);

    return 0;
}
```

Execution Output and Verification (Main Thread Termination)

```
user@machine PRASIE % ./T2-03b
[THREAD 123145405792256] I'm alive!
[THREAD 123145406865408] I'm alive!
[THREAD 123145407401984] I'm alive!
[MAIN THREAD 140704388616576] Main task completed. Exiting.
[THREAD 123145406328832] I'm alive!
[THREAD 123145407938560] I'm alive!
user@machine PRASIE %
```

Thread Argument Passing

Passing data to threads in C requires diligence. Key challenges include:

- **Non-deterministic Scheduling:** The OS scheduler determines execution order; you cannot assume a specific timing.
- **Pointer Reliance:** Arguments are passed via `void*` pointers. If the pointed-to memory changes, the thread receives corrupted data.
- **Memory Lifetime:** Data passed to a thread must remain valid for the entire duration of the thread's execution.

The Common Pitfall: The Loop Index

A frequent error is passing the address of a loop variable directly.

```
for (int i = 0; i < N; i++) {  
    // Incorrect: Every thread points to the same memory location  
    pthread_create(&tid[i], NULL, worker, (void*)&i);  
}
```

- The `i` variable is updated by the *main* thread before the worker thread dereferences the pointer.
- Result: All threads likely read the same final value of `i`.

The Solution: Argument Arrays

To guarantee data integrity, isolate the memory used by each thread.

- **Argument Arrays:** Allocate an array of data where each index corresponds to a specific thread.
- **Isolation:** By passing the address of an array element (&args[i]), each thread receives a unique memory location.
- **Consistency:** The main thread effectively “hands off” that slice of memory to the worker.

Implementation Example

```
int args[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++) {
    args[i] = i; // Unique value per thread
    pthread_create(&tid[i], NULL, task, (void*)&args[i]);
}

// Ensure threads complete before 'args' goes out of scope
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(tid[i], NULL);
}
```

Best Practices Checklist

Ensure thread safety by verifying:

- **Memory Isolation:** Does each thread have its own dedicated address for its input?
- **Lifetime Management:** Is the argument's scope persisting until the thread finishes?
- **Joinability:** Are you using `pthread_join()` to ensure the main thread waits for data consumption?
- **Type Safety:** Are pointers correctly cast from `void*` and dereferenced safely?

Passing Multiple Arguments: Structures

The `pthread_create()` function accepts only a single `void*` argument. To pass multiple parameters, we must encapsulate them:

- **Encapsulation:** Group diverse data types (integers, strings, pointers) into a single struct.
- **Type Safety:** Cast the `void*` back to a pointer of your specific structure type inside the thread function.
- **Organisation:** This approach scales effectively for complex data and improves code maintainability.

Implementing Struct Arguments 1/2

```
typedef struct {
    int thread_id;
    double threshold;
    char log_file[32];
} thread_data_t;

void* worker(void* arg) {
    // Cast back to the original structure type
    thread_data_t *data = (thread_data_t *)arg;
    printf("Thread %d processing %s\n", data->thread_id, data->log_file);
    return NULL;
}

// continued on the next slide...
```

Implementing Struct Arguments 2/2

\\ ... continued from previous slide.

```
int main() {
    pthread_t tid[5];
    thread_data_t pool[5]; // Array of structs for isolation
    for (int i = 0; i < 5; i++) {
        pool[i].thread_id = i;
        sprintf(pool[i].log_file, "log_%d.txt", i);
        pthread_create(&tid[i], NULL, worker, (void *)&pool[i]);
    }
    for (int i = 0; i < 5; i++) pthread_join(tid[i], NULL);
    return 0;
}
```

Managing Structure Lifetime

Memory management is critical when passing pointers to structures:

- **Stack Allocation:** Using a local array in `main()` is safe only if `main()` waits for all threads (via `pthread_join()`) before the function scope ends.
- **Heap Allocation:** If threads must outlive the creating function, use dynamic allocation:
 - ▶ `thread_data_t *d = malloc(sizeof(thread_data_t));`
 - ▶ The worker thread is then responsible for calling `free(arg)` before exiting.
- **Global array:** The best choice for embedded systems.
- **The "Hand-off" Principle:** Once a pointer is passed, the main thread should not modify that memory until the worker has finished with it.